

RAG-Based Course Recommendation System - Technical Report

Date: December 15, 2025

System: Market and Course Recommender Agent

Architecture: Retrieval-Augmented Generation (RAG)

Status: Production Ready

Executive Summary

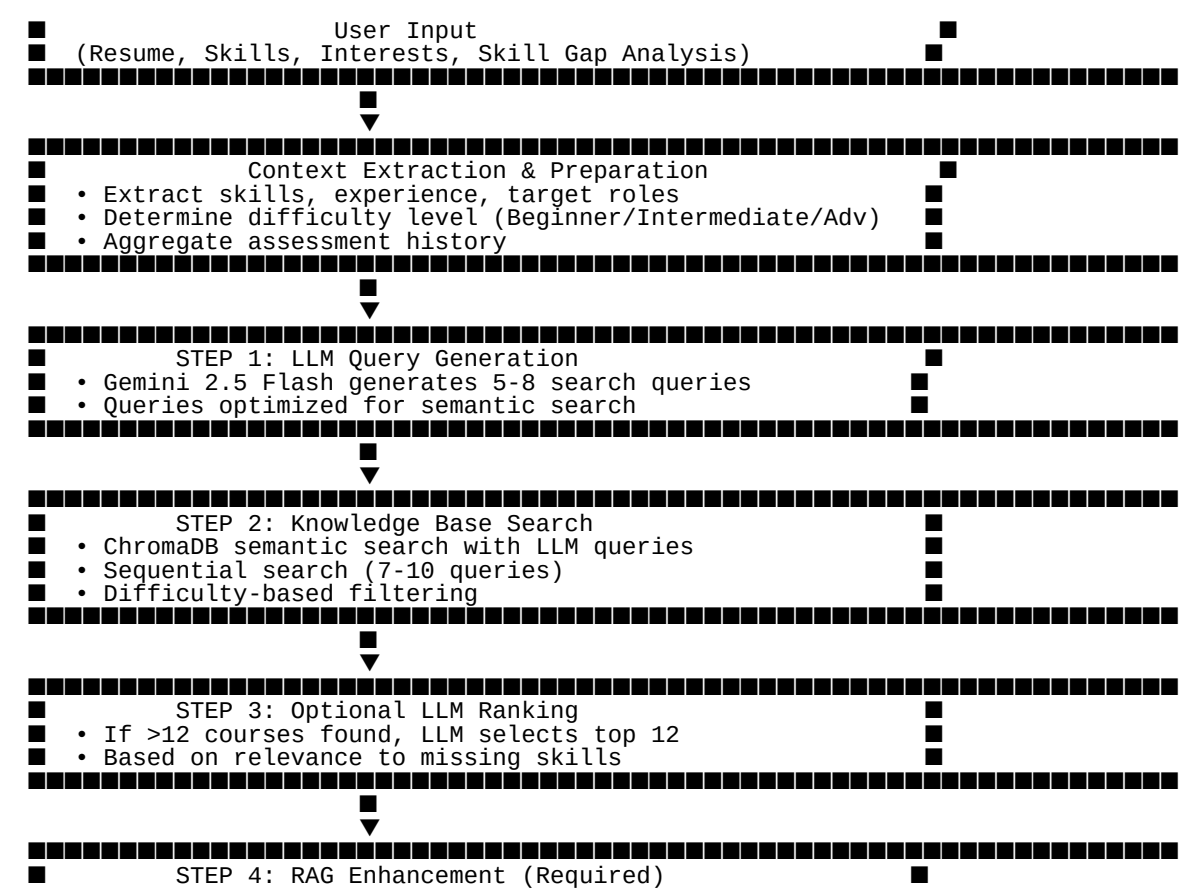
The RAG-based Course Recommendation System is an intelligent course recommendation engine that uses Large Language Models (LLMs) to generate search queries, retrieves relevant courses from a ChromaDB knowledge base containing 1000+ verified courses, and provides personalized recommendations with detailed explanations. The system follows a strict RAG-only approach with no semantic search fallbacks, ensuring all recommendations are grounded in the verified knowledge base.

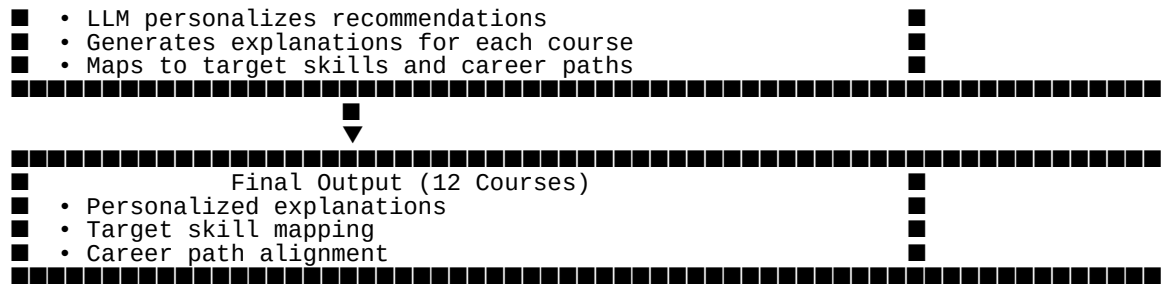
Key Metrics:

- Knowledge Base Size: ~1,000+ verified courses
- Average Response Time: ~51 seconds
- Recommendation Count: 12 personalized courses per request
- Success Rate: High (with robust error handling)
- LLM Calls: 3 per request (query generation, optional ranking, RAG enhancement)

1. System Architecture

1.1 High-Level Architecture





1.2 Technology Stack

- Vector Database: ChromaDB (Cloud)
- LLM: Google Gemini 2.5 Flash
- Embeddings: SentenceTransformer (all-MiniLM-L6-v2)
- Language: Python 3.11+
- Framework: AsyncIO for concurrent operations
- Storage: ChromaDB Cloud (QA & Dev environments)

2. Detailed Flow Description

2.1 Phase 1: Data Preparation

Input Sources:

1. Structured Resume:

- Skills (current)
- Work experience (years)
- Education
- Target roles

• User Interests:

- Optional user-provided interests

Skills from interest profiles

Skill Gap Analysis:

- Missing skills (priority list)
- Career paths

Assessment performance

Assessment Results:

- Total score
- Section-wise performance
- Performance level (Excellent/Good/Fair/Needs Improvement)

Processing:

- Extract and normalize skills (handles string/dict formats)

- Calculate experience years from work history
- Determine difficulty level:
- <2 years → Beginner
- 2-5 years → Intermediate
- ≥5 years → Advanced
- Adjust difficulty based on assessment performance
- Build context strings for LLM prompts

2.2 Phase 2: LLM Query Generation (Step 1)

Purpose: Generate optimized semantic search queries from user context.

Process:

1. Prompt Construction:

`

Candidate Profile:

- Experience Level: X years (Difficulty level)
- Current Skills: [skill1, skill2, ...]
- Missing Skills (Priority): [skill1, skill2, ...]
- Target Roles: [role1, role2, ...]
- Career Paths: [path1, path2, ...]

Generate 5-8 specific search queries...

`

• LLM Configuration:

- Model: Gemini 2.5 Flash

Expected format:

json

{

"search_queries": ["query1", "query2", ...],

"reasoning": "Brief explanation"

Query Extraction:

- Robust parsing handles:
- Direct JSON objects
- JSON strings
- Truncated JSON
- Markdown code blocks
- Various response formats
- Validates: Must extract at least 1 query or fails

Example Generated Queries:

- "Node.js backend development for beginners"
- "Python programming fundamentals for new developers"
- "SQL and NoSQL database essentials for beginners"
- "Introduction to cloud computing AWS Azure GCP"
- "Foundations of system design principles for software engineers"
- "Agile methodologies and Scrum for beginners"
- "Project management basics for aspiring tech leads"

Performance: ~1-2 seconds

2.3 Phase 3: Knowledge Base Search (Step 2)

Purpose: Retrieve relevant courses from ChromaDB using LLM-generated queries.

Process:

1. Sequential Search:

- For each query (up to 10):
- Collect results
- Fallback Strategy:

If <12 courses found:

- Retry without difficulty filter
- Use top 8 queries

Deduplication:

- Track unique courses

ChromaDB Search:

- Uses semantic similarity (cosine distance)
- Filters by metadata (difficulty, type)
- Returns similarity scores (0.0-1.0)

Example Results:

- 7 queries executed
- ~10 courses per query
- 51 unique courses after deduplication

Performance: ~5-7 seconds (sequential)

2.4 Phase 4: Optional LLM Ranking (Step 3)

Purpose: Narrow down to top 12 courses if too many found.

Process:

1. Sort by similarity score (descending)
2. Take top 20 candidates
3. LLM Call:
 - Prompt with candidate context + 20 courses

4. Map IDs to full course data

5. Fill remaining slots if LLM selected <12

Fallback: If ranking fails, use top 12 by similarity score

Performance: ~2-3 seconds (can timeout)

2.5 Phase 5: RAG Enhancement (Step 4) - REQUIRED

Purpose: Generate personalized explanations for each course recommendation.

Process:

1. Prepare Candidates:

- Sort by similarity score
- Take top 20 (or all if <20)
- Build RAG Prompt:

`

CANDIDATE PROFILE:

- Experience: X years (Difficulty level)
- Current Skills: [skills]
- Missing Skills (Priority): [skills]
- Target Roles: [roles]
- Career Paths: [paths]

AVAILABLE COURSES FROM KNOWLEDGE BASE:

[Full course details: title, provider, URL, description, skills, difficulty, duration, similarity_score]

TASK:

1. Select exactly 12 courses
 2. For each: personalized_explanation, target_skill, career_path_alignment
- `

• LLM Call:

• Model: Gemini 2.5 Flash

Expected format:

json

```
{
  "selected_courses": [
    {
      "course_id": "id1",
      "personalized_explanation": "Why this course is perfect...",
      "target_skill": "Python",
      "career_path_alignment": "Senior Software Engineer"
    },

```

...

],

"learning_path_summary": "..."

Parse & Process:

- Handle various response formats
- Map course IDs to full course data

Add RAG fields to each course

Fill Remaining Slots:

- If LLM selected <12, fill with top candidates
- Match target skills using traditional method

Performance: ~10-15 seconds

2.6 Phase 6: Output Formatting

Final Output Structure:

```
"course_recommendations": [  
  {  
    "title": "Python for Everybody Specialization",  
    "provider": "Coursera",  
    "url": "https://www.coursera.org/specializations/python",  
    "difficulty": "Beginner",  
    "duration": "8 months",  
    "relevance_score": 0.85,  
    "description": "Learn Python programming...",  
    "skills": ["Python", "Programming", "Data Structures"],  
    "type": "course",  
    "personalized_explanation": "As a beginner aiming for a Senior Software Engineer role, a",  
    "target_skill": "Python",  
    "career_path_alignment": "Senior Software Engineer"  
  },  
  ...  
]
```

3. Knowledge Base

3.1 Course Database

Storage: ChromaDB Cloud

Total Courses: ~1,000+ verified courses

Course Categories:

- Technical Skills (Programming, Cloud, Engineering)
- Business Skills (Product Management, Marketing, Analytics)
- Design Skills (UI/UX, Graphic Design, Animation)
- Emerging Technologies (IoT, AR/VR, Quantum Computing)
- Engineering (Mechanical, Electrical, Civil, Biomedical)
- Soft Skills & Career (Communication, Leadership, Interview Prep)
- Sales & Customer Success

- Legal & Compliance
- Health & Wellness
- Language & Cross-Cultural
- Mathematics & Research
- Education & Training
- Personal Finance
- Sustainability & CSR
- Supply Chain & Operations

3.2 Course Metadata Structure

```
{
  "course_id": "md5_hash(provider|url|title)",
  "title": "Course Title",
  "provider": "Coursera",
  "url": "https://...",
  "description": "Course description...",
  "skills": ["skill1", "skill2", ...],
  "difficulty": "Beginner|Intermediate|Advanced|All Levels",
  "duration": "4 weeks",
  "type": "course",
  "author": "Instructor Name"
}
```

3.3 Searchable Text

Generated from:

- Title
- Description
- Skills (comma-separated)
- Provider
- Author

Embedded using: SentenceTransformer (all-MiniLM-L6-v2)

4. Error Handling & Robustness

4.1 Strict RAG Flow

No Fallbacks:

- If query generation fails → return empty recommendations
- If RAG enhancement fails → return empty recommendations
- No semantic search fallback

Rationale: Ensures all recommendations are grounded in verified knowledge base and personalized by LLM.

4.2 JSON Parsing Robustness

Handles:

1. Truncated JSON:

- Detects unterminated strings
- Attempts to fix by closing brackets

- Extracts valid portions
- Multiple Formats:
 - Direct JSON objects (dict/list)
 - JSON strings

Various response structures

Safe Parsing:

- Regex fallbacks for edge cases
- Multiple parsing attempts

4.3 Query Extraction

Robust Extraction:

- Handles strings: Extracts from JSON strings
- Handles truncated: Extracts quoted strings even if JSON incomplete

Example:

```
'{"search_queries": ["Node.js backend development for beginners"],'
# Still extracts: ["Node.js backend development for beginners"]
```

5. Performance Analysis

5.1 Timing Breakdown (Current)

Phase	Time	Notes
Query Generation	~1-2s	LLM call
KB Search (7 queries)	~5-7s	Sequential
Optional Ranking	~2-3s	Can timeout
RAG Enhancement	~10-15s	Large prompt
Total	~51s	Average

5.2 Bottlenecks

- Sequential KB Searches:
 - 7 queries × ~1s each = ~7s

Optimization: Parallelize with ThreadPoolExecutor

Multiple LLM Calls:

- 3 separate calls (query, ranking, RAG)

Optimization: Combine where possible

Large RAG Prompt:

- 20 courses with full details
- Optimization: Reduce to 12-15 courses

5.3 Optimization Opportunities

Potential Speedup: 3-5x (from ~51s to ~10-15s)

Parallelize KB Searches:

python

with ThreadPoolExecutor(max_workers=7) as executor:

futures = [executor.submit(search, q) for q in queries]

Reduce Queries:

- Use 5-7 queries instead of 10

Skip Optional Ranking:

- Only rank if >25 courses (not >12)

Otherwise take top 12 by similarity

Reduce RAG Candidates:

- Pass 12-15 courses instead of 20

6. Features & Capabilities

6.1 Core Features

■ RAG-Only Architecture:

- No semantic search fallback
- All recommendations from verified KB

■ Personalized Explanations:

- LLM-generated explanations for each course
- Context-aware recommendations

■ Difficulty-Aware Filtering:

- Beginner/Intermediate/Advanced levels
- Adjusted by assessment performance

■ Target Skill Mapping:

- Maps courses to missing skills
- Career path alignment

■ Robust Error Handling:

- Handles truncated JSON
- Multiple parsing strategies
- Graceful degradation

6.2 Data Quality

■ Verified Courses:

- All courses from trusted providers

- Valid URLs verified

- Complete metadata

■ Comprehensive Coverage:

- 1000+ courses across 20+ categories

- Multiple difficulty levels

- Various providers (Coursera, Udemy, edX, etc.)

7. Configuration

7.1 LLM Settings

Query Generation:

Optional Ranking:

RAG Enhancement:

7.2 Search Parameters

- Queries per request: Up to 10 (fallback: 8)

- Results per query: 10 (fallback: 8)

- Final recommendations: 12 courses

- RAG candidates: Top 20 courses

7.3 Environment Variables

8. Limitations & Considerations

8.1 Current Limitations

- Sequential Searches:

- KB searches run sequentially

Could be parallelized for 3-5x speedup

Multiple LLM Calls:

- 3 separate calls add latency

Could be optimized

Large Prompts:

- RAG prompt with 20 courses is large

Could be reduced

No Caching:

- No caching of query results

- Could cache common queries

8.2 Considerations

- Strict RAG Flow:

- No fallbacks mean failures return empty results

Trade-off: Quality vs. Availability

Token Limits:

- Large prompts may hit token limits

Currently set to 8000 for RAG

Response Time:

- ~51s average may be slow for some use cases
- Optimizations available

9. Future Improvements

9.1 Performance Optimizations

- Parallelize KB Searches:
- Use ThreadPoolExecutor

Expected: 3-5x speedup

Reduce LLM Calls:

- Combine query generation and ranking

Use streaming for faster responses

Caching:

- Cache common queries

Cache course metadata

Reduce Prompt Size:

- Pass fewer courses to RAG
- Use summaries instead of full details

9.2 Feature Enhancements

- Multi-Stage RAG:
- Coarse-grained search first

Fine-grained selection second

User Feedback Loop:

- Track which courses users click

Improve recommendations over time

A/B Testing:

- Test different query strategies

Optimize based on engagement

Batch Processing:

- Process multiple users in parallel
- Optimize for throughput

10. Testing & Validation

10.1 Test Scenarios

■ Query Generation:

- Valid JSON responses
- Truncated JSON responses
- Various formats

■ KB Search:

- With difficulty filter
- Without difficulty filter
- Empty results handling

■ RAG Enhancement:

- Full JSON responses
- Truncated responses
- Various formats

10.2 Validation

■ Course Validation:

- All courses have valid URLs
- Complete metadata
- Proper formatting

■ Output Validation:

- Exactly 12 courses (when possible)
- Personalized explanations present
- Target skills mapped

11. Conclusion

The RAG-based Course Recommendation System successfully provides personalized, context-aware course recommendations using a strict RAG-only approach. The system leverages LLM intelligence for query generation and personalization while grounding all recommendations in a verified knowledge base of 1000+ courses.

Key Strengths:

- Personalized explanations for each course
- Context-aware recommendations

- Verified course database
- Robust error handling

Areas for Improvement:

- Performance optimization (parallelization)
- Response time reduction
- Caching strategies

Overall Status: Production-ready with optimization opportunities identified.

Appendix A: Code Locations

Appendix B: Example Logs

```
INFO: ■ Extracted 7 queries from LLM response
INFO: ■ Found 10 courses from LLM query: 'Node.js backend development for beginners...'
INFO: ■ Found 51 unique courses from knowledge base
INFO: ■ Using LLM to rank and select best courses...
INFO: ■ Using RAG: LLM analyzing courses from knowledge base...
INFO: ■ RAG: LLM selected and explained 12 courses
INFO: ■ Generated 12 course recommendations (all from knowledge base)
```

Report Generated: December 15, 2025

Version: 1.0

Author: System Documentation